

AGRITECH RESEARCH AND STUDY CLUB (ARSC)

Divisi Pengembangan Sumber Daya Manusia | Periode 2025/2026

CONCEPT PAPER

SISTEM HALO PSDM BERBASIS WEBSITE

Tech Stack

Next.js 16 · Bun Runtime · Vercel Serverless · Supabase (PostgreSQL) · Pusher Channels

Dokumen Perencanaan Sistem

Divisi PSDM ARSC | Maret 2025 | Versi 1.0

Daftar Isi

1. Ringkasan Eksekutif.....	3
2. Latar Belakang dan Konteks Masalah	4
2.1 Permasalahan yang Diidentifikasi	4
2.2 Solusi yang Ditawarkan.....	4
3. Tujuan dan Sasaran Pengembangan	5
3.1 Tujuan Utama	5
3.2 Indikator Keberhasilan.....	5
4. Daftar Fitur Lengkap.....	6
4.1 Fitur Sender	6
4.2 Fitur Admin	6
4.3 Fitur Sistem Otomatis.....	7
5. Arsitektur Sistem	7
5.1 Gambaran Umum Arsitektur Serverless	7
5.2 Komponen Teknologi Stack	8
5.3 Skema Database Supabase	8
6. Struktur Akses dan Peran (Role System)	9
7. Alur Sistem Detail dan Business Logic	10
7.1 Alur Pelaporan / Pengaduan Masalah	10
7.2 Alur Cerita / Curhat (Live Chat System)	14
7.3 Alur Permintaan Janji Temu	17
8. Struktur API Routes (Serverless Functions)	19
9. Sistem Notifikasi	20
10. Strategi Deployment di Vercel	21
11. Monitoring, Evaluasi, dan Rekap Akhir Periode	22
12. Pertimbangan Keamanan dan Privasi.....	23
13. Roadmap Implementasi	24
14. Penutup	24

1. Ringkasan Eksekutif

Halo PSDM adalah sistem komunikasi dua arah berbasis website yang dirancang untuk Divisi Pengembangan Sumber Daya Manusia (PSDM) Agritech Research and Study Club (ARSC) periode 2025/2026. Sistem ini lahir dari evaluasi periode sebelumnya yang mengidentifikasi bahwa alur pelaporan yang terlalu panjang dan bertahap menyebabkan lambatnya penanganan kasus serta terlambatnya identifikasi masalah yang berpotensi mengganggu ekosistem organisasi.

Secara teknis, sistem dibangun di atas stack modern yang sepenuhnya kompatibel dengan lingkungan serverless: Next.js 16 (App Router) sebagai framework utama, Bun sebagai JavaScript runtime dan package manager, Vercel sebagai platform deployment, serta Supabase sebagai backend-as-a-service yang menyediakan database PostgreSQL, Row Level Security, dan realtime subscription. Pendekatan serverless dipilih secara strategis untuk menjamin skalabilitas, biaya operasional minimal, dan kemudahan deployment tanpa memerlukan pengelolaan server secara manual.

Tiga fitur utama sistem:

- **Pelaporan / Pengaduan Masalah** — sistem tiket terstruktur dengan Case ID unik, status lifecycle, dan notifikasi berlapis.
- **Cerita / Curhat (Live Chat)** — komunikasi asinkron dan real-time yang dimodelkan seperti Halodoc, dengan arsip penuh di database.
- **Permintaan Janji Temu** — direktori kontak admin dengan redirect langsung ke WhatsApp dan nomor yang tersimpan aman.

2. Latar Belakang dan Konteks Masalah

2.1 Permasalahan yang Diidentifikasi

Berdasarkan evaluasi internal ARSC pada akhir periode sebelumnya, ditemukan beberapa permasalahan sistemik dalam proses penanganan laporan dan pendampingan anggota:

Permasalahan	Akar Penyebab	Dampak pada Organisasi
Alur pelaporan terlalu panjang dan bertahap	Tidak ada kanal terpusat; laporan harus melewati banyak pihak sebelum sampai ke PH PSDM	Penanganan lambat, masalah kecil berkembang menjadi konflik yang lebih besar
Tidak ada sistem tiket atau nomor kasus	Pelaporan dilakukan via chat personal yang tidak terstruktur	Sulit memonitor status dan riwayat; kasus mudah terlewat atau terlupa
Komunikasi terfragmentasi di berbagai platform	WhatsApp pribadi digunakan sebagai kanal utama pelaporan	Arsip tidak terpusat, rawan kehilangan data di pergantian pengurus

Permasalahan	Akar Penyebab	Dampak pada Organisasi
Tidak ada indikator ketersediaan admin	Tidak ada sistem yang menampilkan status online/offline PH PSDM	Anggota tidak tahu kapan laporan akan direspons; frustrasi meningkat
Proses janji temu tidak terstruktur	Permintaan janji temu dilakukan ad-hoc via pesan personal	Admin PSDM sulit mengelola jadwal; permintaan sering terlupakan

2.2 Solusi yang Ditawarkan

Halo PSDM hadir sebagai solusi digital terpusat yang mengintegrasikan seluruh kanal komunikasi antara anggota ARSC dengan PH PSDM ke dalam satu platform website. Dengan pendekatan ini, setiap laporan mendapat nomor tiket unik, setiap sesi chat tersimpan sebagai arsip yang dapat diaudit, dan seluruh aktivitas terdokumentasi secara sistematis untuk keperluan monitoring dan evaluasi akhir periode. Penggunaan Supabase sebagai backend memungkinkan implementasi Row Level Security (RLS) langsung di level database, sehingga keamanan data anggota terjamin tanpa perlu logika filtering manual di setiap API endpoint.

3. Tujuan dan Sasaran Pengembangan

3.1 Tujuan Utama

1. **Komunikasi Terpusat & Aman:** Menyediakan ruang komunikasi dua arah yang aman, responsif, dan terdokumentasi antara anggota ARSC dengan PH PSDM, menggantikan kanal personal yang tidak terstruktur.
2. **Efektivitas Penanganan:** Meningkatkan kecepatan dan akurasi penanganan laporan serta pendampingan anggota melalui digitalisasi alur, sistem tiket, dan status tracking.
3. **Human Relations Terstruktur:** Mewujudkan sistem human relations dengan rekam jejak kasus yang dapat diaudit oleh pengurus periode berikutnya.
4. **Integrasi Janji Temu:** Mengintegrasikan sistem permintaan janji temu secara langsung dengan kontak WhatsApp, dengan pencatatan untuk keperluan rekap.
5. **Keberlanjutan Sistem:** Membangun sistem yang mudah di-maintain dan dapat diwariskan lintas periode tanpa ketergantungan pada infrastruktur server yang kompleks.

3.2 Indikator Keberhasilan

Indikator	Target	Cara Pengukuran
Response time rata-rata laporan	< 24 jam sejak laporan masuk	Selisih created_at vs first status update di DB
Tingkat penyelesaian kasus	> 90% berstatus DONE di akhir periode	Query COUNT reports WHERE status = DONE

Indikator	Target	Cara Pengukuran
Uptime sistem	> 99.5%	Vercel Analytics + Uptime monitoring
Arsip sesi chat tersimpan	100% sesi tercatat di database	Audit tabel chat_sessions vs chat_messages
Kepuasan anggota	> 80% rating positif	Survei Google Form akhir periode

4. Daftar Fitur Lengkap

Berikut adalah inventarisasi fitur lengkap sistem Halo PSDM, dibagi berdasarkan aktor yang menggunakannya.

4.1 Fitur Sender (PH, Staf Ahli, Anggota Muda)

Kode Fitur	Nama Fitur	Deskripsi
F-S01	Login & Manajemen Akun	Login menggunakan username/password. Data profil (nama, biro, jabatan) tersinkron otomatis dari akun dan tampil di form pelaporan.
F-S02	Kirim Laporan / Pengaduan	Form pengaduan dengan field: kategori masalah (dropdown), kronologi (textarea), dan tingkat urgensi. Submit menghasilkan Case ID unik.
F-S03	Lacak Status Laporan	Dashboard pribadi menampilkan semua laporan yang pernah dikirim beserta status terkini (RECEIVED / IN_PROGRESS / NEEDS_CLARIFICATION / DONE) secara real-time.
F-S04	Melihat Detail Kasus	Halaman detail per Case ID menampilkan kronologi laporan, riwayat perubahan status, catatan admin, dan thread chat klarifikasi (jika ada).
F-S05	Memulai Sesi Curhat / Chat	Sender dapat membuka sesi chat baru dengan PH PSDM kapan saja. Tersedia indikator status admin (Online/Offline).
F-S06	Kirim & Terima Pesan Real-time	Antarmuka chat dua arah dengan indikator status pesan: Terkirim (✓) dan Dibaca (✓✓). Pesan baru muncul tanpa perlu refresh halaman.
F-S07	Akses Riwayat Chat	Seluruh riwayat percakapan tersimpan dan dapat diakses kembali oleh Sender dari menu 'Riwayat Chat'.
F-S08	Permintaan Janji Temu	Direktori daftar admin (PH PSDM dan HR/Staf Ahli PSDM) dengan tombol redirect langsung ke WhatsApp.

Kode Fitur	Nama Fitur	Deskripsi
F-S09	Notifikasi In-app & Email	Notifikasi in-app (badge dan pop-up) untuk update status laporan dan balasan chat. Email fallback jika Sender sedang offline.
F-S10	Pengiriman Laporan Anonim (Opsional)	Sender dapat memilih untuk mengirim laporan secara anonim. Identitas tersimpan di DB (untuk audit admin senior) tetapi tidak ditampilkan ke admin penerima tanpa izin.

4.2 Fitur Admin (PH PSDM)

Kode Fitur	Nama Fitur	Deskripsi
F-A01	Dashboard Ringkasan Kasus	Halaman utama admin menampilkan: jumlah laporan aktif, laporan belum ditangani, sesi chat terbuka, dan permintaan janji temu. Dilengkapi badge counter real-time.
F-A02	Notifikasi Real-time	Pop-up notifikasi langsung saat ada laporan baru, chat baru, atau permintaan janji temu. Dikirim via Pusher Channels. Badge counter bertambah secara otomatis.
F-A03	Manajemen Kasus Laporan	Tabel daftar semua kasus dengan filter (status, biro, tanggal) dan sorting. Admin dapat membuka detail, menambah catatan internal, dan mengubah status kasus.
F-A04	Update Status Kasus	Dropdown status: RECEIVED → IN_PROGRESS → NEEDS_CLARIFICATION → DONE. Setiap perubahan status otomatis mengirim notifikasi ke Sender.
F-A05	Catatan Internal Kasus	Field catatan yang hanya terlihat oleh sesama admin, tidak terlihat oleh Sender. Berguna untuk koordinasi internal tim PSDM.
F-A06	Balasan Chat Real-time	Antarmuka chat admin yang terhubung dengan session Sender. Admin dapat membalas dari halaman detail kasus atau dari menu Chat. Semua percakapan tersimpan otomatis.
F-A07	Kelola Status Ketersediaan	Admin dapat mengubah status ketersediaan (Online/Away/Offline) yang ditampilkan ke Sender di halaman chat.
F-A08	Arsip Kasus & Chat	Semua kasus dan sesi chat yang sudah ditutup tersimpan sebagai arsip read-only dan dapat dicari/difilter kapan saja.
F-A09	Manajemen Data Admin	CRUD data admin (nama, jabatan, nomor WA) untuk keperluan direktori janji temu. Hanya dapat diakses oleh Super Admin (Kepala Divisi PSDM).

Kode Fitur	Nama Fitur	Deskripsi
F-A10	Laporan Rekap & Analitik	Dashboard monitoring: grafik laporan per periode, distribusi kategori masalah, average response time, dan rekap janji temu.

4.3 Fitur Sistem Otomatis

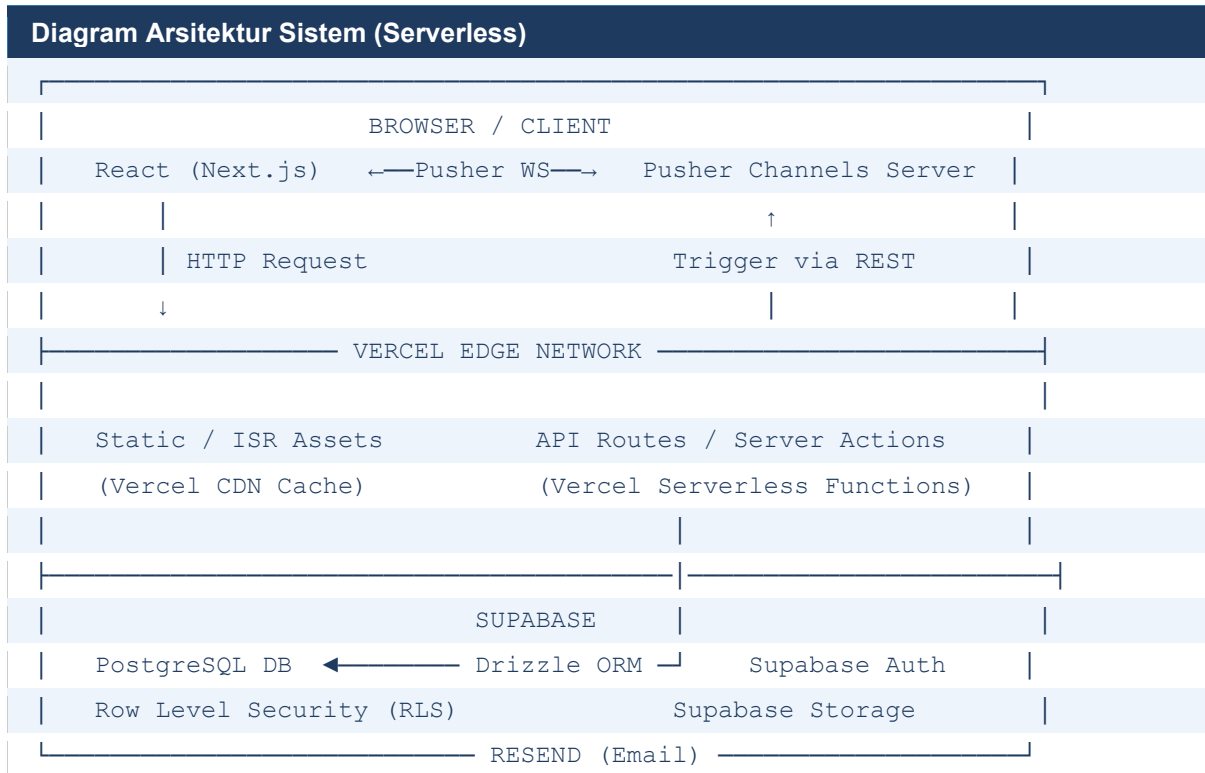
Kode Fitur	Nama Fitur	Trigger & Aksi
F-SYS01	Generate Case ID	Trigger: laporan baru di-submit. Aksi: sistem membuat ID unik dengan format HP-[YYYYMMDD]-[4 digit random], memastikan tidak ada duplikat via DB constraint.
F-SYS02	Email Konfirmasi Sender	Trigger: laporan berhasil disimpan ke DB. Aksi: Resend API mengirim email ke Sender dengan Case ID dan ringkasan laporan.
F-SYS03	Email Notifikasi Admin	Trigger: laporan/chat baru masuk saat admin offline. Aksi: Resend API mengirim email digest ke semua admin aktif.
F-SYS04	Pusher Event Broadcast	Trigger: setiap aksi baru (laporan, pesan, update status). Aksi: Serverless Function memanggil Pusher Server SDK untuk broadcast ke channel terkait.
F-SYS05	WhatsApp URL Generator	Trigger: Sender klik nama admin di menu janji temu. Aksi: API Route mengambil nomor WA dari DB (terenkripsi), men-decode, dan mengembalikan URL wa.me yang sudah diformat.
F-SYS06	Auto-close Chat Session	Trigger: status laporan berubah menjadi DONE (jika ada session chat terkait). Aksi: chat_session otomatis di-update ke CLOSED, Sender mendapat notifikasi.
F-SYS07	Rate Limiting	Trigger: request berlebihan dari IP yang sama ke endpoint sensitif. Aksi: Vercel KV menghitung request count; jika melebihi limit, kembalikan 429 Too Many Requests.

5. Arsitektur Sistem

5.1 Gambaran Umum Arsitektur Serverless

Sistem Halo PSDM mengadopsi arsitektur serverless full-stack berbasis Next.js 16 App Router. Dalam model ini, tidak ada server yang selalu aktif (always-on). Setiap permintaan HTTP ditangani oleh Vercel Serverless Functions yang di-spawn secara dinamis, sehingga biaya hanya dibebankan berdasarkan jumlah eksekusi aktual. Tantangan utama arsitektur ini — yaitu

WebSocket yang tidak bisa di-maintain oleh stateless function — diatasi dengan Pusher Channels sebagai managed WebSocket broker eksternal.



5.2 Komponen Teknologi Stack

Komponen	Teknologi	Justifikasi Pemilihan
Framework	Next.js 16 (App Router)	Server Components, Server Actions, dan API Routes semuanya di-deploy sebagai Serverless Functions di Vercel secara otomatis. Zero-config untuk deployment.
Runtime & Package Manager	Bun	Pengganti Node.js dan npm; install dependencies hingga 10x lebih cepat, eksekusi script lebih efisien, dan kompatibel penuh dengan ekosistem Node.js.
Deployment Platform	Vercel	Zero-config deployment untuk Next.js, auto-scaling global, preview deployments per pull request, Vercel Analytics, dan Vercel KV untuk caching.
Database & Backend	Supabase (PostgreSQL)	Managed PostgreSQL dengan Row Level Security (RLS) built-in, connection pooling via Supavisor (HTTP mode, cocok untuk serverless), realtime subscription, dan Supabase Storage untuk file.
ORM	Drizzle ORM	Type-safe, ringan, mendukung edge runtime Vercel, dan memiliki integrasi langsung dengan Supabase connection string.
Real-time Messaging	Pusher Channels	Managed WebSocket broker; cocok untuk serverless karena state koneksi dikelola di luar

Komponen	Teknologi	Justifikasi Pemilihan
		Next.js function. Serverless Function hanya memanggil Pusher REST API untuk trigger event.
Email Notifikasi	Resend	Transactional email API modern dengan React Email template support. Free tier cukup untuk kebutuhan organisasi skala ARSC.
Autentikasi	NextAuth.js v5 (Auth.js)	Session management dengan JWT, mendukung credential provider untuk login berbasis username/password yang di-verifikasi ke Supabase.
Styling	Tailwind CSS v4	Utility-first, zero-runtime CSS overhead, dan kompatibel sempurna dengan Next.js App Router Server Components.
Caching & Rate Limit	Vercel KV (Redis)	Untuk rate limiting endpoint sensitif, cache data admin directory, dan temporary session state.

5.3 Skema Database Supabase

Berikut adalah skema tabel utama di Supabase PostgreSQL, beserta kebijakan Row Level Security (RLS) yang diterapkan:

Tabel	Kolom Utama	RLS Policy
users	id (UUID), name, biro, jabatan, role (SENDER/ADMIN/SUPER_ADMIN), email, password_hash, is_active, created_at	SELECT: user hanya bisa baca data dirinya sendiri. Admin bisa baca semua.
reports	id (UUID), case_id (UNIQUE: HP-YYYYMMDD-XXXX), user_id (FK), category, urgency, kronologi, is_anonymous, status (ENUM), admin_notes, created_at, updated_at	SELECT: Sender hanya bisa baca report miliknya. Admin bisa baca semua. INSERT: hanya SENDER role.
report_status_history	id, report_id (FK), old_status, new_status, changed_by (admin_id), note, created_at	SELECT: Admin bisa baca semua. Sender hanya bisa baca history laporan miliknya.
chat_sessions	id (UUID), report_id (FK, nullable), user_id (FK), assigned_admin_id (FK, nullable), status (OPEN/CLOSED), created_at, closed_at	SELECT: Sender hanya bisa baca session miliknya. Admin bisa baca semua.
chat_messages	id (UUID), session_id (FK), sender_id (FK), content, is_read, read_at, created_at	INSERT: hanya jika user adalah member session. SELECT: sama dengan chat_sessions.

Tabel	Kolom Utama	RLS Policy
appointments	id, user_id (FK), target_admin_id (FK), wa_redirect_logged_at, created_at	INSERT: hanya SENDER. SELECT: Admin bisa baca semua untuk rekap.
admin_profiles	id (FK ke users.id), display_name, jabatan_display, wa_number_encrypted, availability_status, avatar_url, updated_at	SELECT: semua authenticated user bisa baca (kecuali wa_number_encrypted). UPDATE: hanya ADMIN dirinya sendiri atau SUPER_ADMIN.
notifications	id, user_id (FK), type (ENUM), payload (JSONB), is_read, created_at	SELECT: user hanya bisa baca notifikasi miliknya sendiri.

6. Struktur Akses dan Peran (Role System)

Sistem mengimplementasikan Role-Based Access Control (RBAC) dengan tiga role yang tersimpan di kolom role tabel users:

Role	Pengguna	Hak Akses Utama
SENDER	PH (non-PSDM), Staf Ahli, Anggota Muda	Akses ke /dashboard, /laporan, /chat, /appointments. Hanya bisa baca/kirim data milik diri sendiri (dijamin RLS Supabase).
ADMIN	PH PSDM	Akses ke semua route SENDER plus /admin/*. Bisa membaca semua laporan, membalas chat, update status kasus, dan mengelola profil diri sendiri.
SUPER_ADMIN	Kepala Divisi PSDM	Akses penuh termasuk manajemen akun admin, CRUD data admin_profiles (nomor WA), akses laporan rekap, dan audit log.

Middleware Auth — Berjalan di Vercel Edge Runtime (middleware.ts)

Setiap HTTP request melewati middleware SEBELUM sampai ke route handler atau page.

1. Middleware membaca JWT session token dari cookie HttpOnly (diset oleh NextAuth.js).
2. Jika tidak ada token atau token expired → redirect ke /login.
3. Jika role = SENDER mencoba akses /admin/* → redirect ke /dashboard (403 implicit).
4. Jika role = ADMIN atau SUPER_ADMIN → akses penuh ke /admin/* diizinkan.
5. Di setiap Server Action dan API Route, validasi role dilakukan ULANG di server

(defense in depth) — tidak bergantung hanya pada middleware.

6. Supabase RLS menjadi lapisan keamanan KETIGA: meski query berhasil masuk ke DB, data yang dikembalikan otomatis difilter berdasarkan user yang authenticated.

7. Alur Sistem Detail dan Business Logic

Bagian ini menjabarkan alur lengkap untuk setiap fitur utama, mencakup perspektif pengguna (Sender), logika bisnis di server (Serverless Functions), dan respons admin. Setiap langkah dirancang untuk berjalan sepenuhnya di atas infrastruktur serverless Vercel.

7.1 Alur Pelaporan / Pengaduan Masalah

Fitur pelaporan adalah inti dari sistem Halo PSDM. Setiap laporan mengikuti lifecycle status yang terdefinisi dari RECEIVED hingga DONE. Setiap transisi status tercatat di tabel `report_status_history` untuk audit trail yang lengkap.

Status Lifecycle Laporan

Status	Artinya	Siapa yang Mengubah	Notifikasi ke Sender
RECEIVED	Laporan masuk dan diterima sistem. Belum ada admin yang menangani.	Otomatis saat submit	Email konfirmasi + Case ID
IN_PROGRESS	Admin sedang menangani dan menginvestigasi laporan.	Admin (manual)	Email + notifikasi in-app
NEEDS_CLARIFICATION	Admin membutuhkan informasi tambahan dari Sender. Chat sesi otomatis dibuka.	Admin (manual)	Email + notifikasi + chat session dibuka
DONE	Kasus dinyatakan selesai oleh admin.	Admin (manual)	Email 'Kasus diselesaikan' + notifikasi in-app

7.1.1 Alur Sender (Pelapor)

#	Aksi	Detail & Business Logic
1	Akses & Login	Sender membuka website Halo PSDM dan login dengan akun ARSC. NextAuth.js memvalidasi credentials dengan query ke tabel users di Supabase. Jika valid, session JWT dibuat dan disimpan di cookie HttpOnly (tidak accessible oleh JavaScript). Session berisi: userId, name, biro, jabatan, role.
2	Buka Menu Pelaporan	Dari /dashboard, Sender klik menu 'Pelaporan / Pengaduan'. Next.js Server Component merender halaman form. Data profil (nama, biro, jabatan) otomatis di-prefill dari session — Sender tidak perlu mengisi ulang. Ini mengurangi friction dan memastikan data identitas konsisten.
3	Pilih Kategori & Isi Kronologi	Sender mengisi form: (a) Kategori Masalah (dropdown: Konflik Antar Anggota / Beban Kerja / Kesejahteraan / Akademik / Lainnya), (b) Tingkat Urgensi (RENDAH / SEDANG / TINGGI), (c) Kronologi Permasalahan (textarea, min. 50 karakter), (d) opsi Kirim Anonim (checkbox). Client-side validation via Zod schema memberikan feedback instan sebelum submit.
4	Submit Laporan	Sender klik 'Kirim Laporan'. Form di-submit via Next.js Server Action (bukan fetch manual ke API). Server Action berjalan sebagai Serverless Function di Vercel — request tidak pernah melalui client-side JavaScript untuk processing sensitif. CSRF protection built-in di Server Actions.
5	Redirect & Konfirmasi	Setelah submit berhasil, Sender di-redirect ke /dashboard/laporan/[caseId] dan melihat toast notifikasi: 'Laporan Anda (HP-YYYYMMDD-XXXX) telah diterima.' Halaman menampilkan status RECEIVED dan estimasi response time.
6	Pantau Status Real-time	Sender dapat memantau status dari /dashboard/laporan kapan saja. Perubahan status muncul secara real-time tanpa refresh via Pusher event subscription di client. Riwayat perubahan status (dari report_status_history) ditampilkan sebagai timeline di halaman detail.

7.1.2 Business Logic di Server (Server Action: submitReport)

Business Logic: submitReport()	
Server Action: submitReport(formData) – Vercel Serverless Function	
STEP 1	Autentikasi & Otorisasi
	→ Ambil session dari NextAuth; jika tidak ada, throw AuthError
	→ Validasi role = SENDER (ADMIN tidak boleh membuat laporan sendiri)

STEP 2	Validasi Input (Server-side Zod)
	→ category: <code>enum['KONFLIK','BEBAN_KERJA','KESEJAHTERAAN','AKADEMIK','LAIN']</code> → urgency: <code>enum['RENDAH','SEDANG','TINGGI']</code> → kronologi: <code>string, min 50 char, max 5000 char</code> → is_anonymous: <code>boolean</code> - Jika gagal → kembalikan <code>validationError</code> ke form (tidak throw exception)
STEP 3	Generate Case ID Unik
	→ Format: <code>HP-{YYYYMMDD}-{4 digit random alphanum}</code> → Cek duplikat di DB; jika bentrok (sangat jarang), regenerate
STEP 4	INSERT ke Supabase
	→ Tabel <code>reports</code>: <code>case_id, user_id, category, urgency, kronologi, is_anonymous</code> → Status awal: <code>RECEIVED</code> → Tabel <code>report_status_history</code>: initial entry (null → <code>RECEIVED</code>) → Tabel <code>notifications</code>: INSERT untuk semua admin aktif (query <code>users WHERE role IN ADMIN</code>)
STEP 5	Trigger Pusher Event (fire-and-forget, tidak block response)
	→ Channel: <code>'private-admin-global'</code> → Event: <code>'new-report'</code> → Payload: <code>{ caseId, senderName (masked jika anonim), category, urgency, createdAt }</code>
STEP 6	Kirim Email via Resend (fire-and-forget)
	→ Email ke Sender: konfirmasi dengan Case ID dan ringkasan laporan → Email ke semua admin aktif: notifikasi laporan baru dengan link ke <code>/admin/laporan/[caseId]</code>
STEP 7	Return ke Server Action Caller
	→ <code>{ success: true, caseId: 'HP-...' }</code> → Next.js otomatis revalidate path <code>'/dashboard/laporan'</code>

7.1.3 Alur Admin (PH PSDM) — Menangani Laporan

#	Aksi	Detail & Business Logic
1	Terima Notifikasi	Admin yang sedang aktif di <code>/admin/dashboard</code> menerima: (a) pop-up notifikasi real-time via Pusher Channels (muncul di pojok kanan atas), (b) badge counter di sidebar menu 'Laporan' bertambah, (c) email notifikasi via Resend (sebagai fallback dan untuk admin yang sedang offline).
2	Buka Daftar Laporan	Admin mengakses <code>/admin/laporan</code> . Server Component me-render tabel laporan yang diambil dari Supabase dengan query: <code>SELECT * FROM reports ORDER BY urgency DESC, created_at ASC</code> . Laporan urgensi TINGGI muncul di atas. Admin dapat filter berdasarkan status, kategori, biro, dan rentang tanggal.

#	Aksi	Detail & Business Logic
3	Buka Detail Laporan	Admin klik laporan untuk membuka /admin/laporan/[caseId]. Halaman menampilkan: identitas pelapor (dengan catatan jika anonim), kronologi lengkap, timeline status, catatan internal sebelumnya, dan tombol aksi update status. Supabase RLS memastikan admin hanya bisa baca laporan yang valid.
4	Update Status Laporan	Admin mengubah status melalui dropdown. Server Action updateReportStatus() dipanggil. Business logic: (a) validasi transisi status valid (tidak boleh langsung dari RECEIVED ke DONE untuk urgensi TINGGI tanpa melewati IN_PROGRESS), (b) INSERT ke report_status_history, (c) trigger Pusher event 'status-updated' ke channel Sender, (d) kirim email notifikasi ke Sender.
5	Tambah Catatan Internal	Admin dapat menambahkan catatan internal yang hanya terlihat oleh sesama admin (disimpan di kolom admin_notes tabel reports). Berguna untuk koordinasi: 'sudah kontak via WA pribadi, menunggu konfirmasi' dll. RLS memastikan Sender tidak bisa membaca kolom ini.
6	Buka Klarifikasi via Chat	Jika admin memilih status NEEDS_CLARIFICATION: Server Action otomatis membuat chat_session baru yang terhubung dengan report_id, mengubah status laporan, dan mengirim notifikasi ke Sender bahwa 'Admin membutuhkan klarifikasi. Sesi chat telah dibuka.' Sender langsung bisa membalas di fitur chat.
7	Selesaikan Kasus	Admin mengubah status ke DONE. Business logic: (a) jika ada chat_session terkait, otomatis close session, (b) INSERT final entry ke status_history, (c) kirim email 'Kasus [Case ID] telah diselesaikan' ke Sender, (d) trigger Pusher event ke Sender. Data kasus tetap tersimpan sebagai arsip read-only yang dapat diaudit.

7.2 Alur Cerita / Curhat (Live Chat System)

Fitur Live Chat dimodelkan seperti sistem chat Halodoc: asinkron dengan indikator ketersediaan admin dan penyimpanan arsip penuh. Tantangan utama dalam arsitektur serverless adalah bahwa Vercel Serverless Functions tidak bisa maintain persistent WebSocket connection karena bersifat stateless dan berumur pendek. Solusi ini diatasi dengan Pusher Channels sebagai managed WebSocket broker yang terpisah dari infrastruktur Next.js.

Arsitektur Koneksi Real-time (Serverless-Safe)

Masalah: Serverless Function tidak bisa maintain WebSocket

Vercel Function hidup hanya selama request berlangsung (~ms hingga beberapa detik).

WebSocket membutuhkan koneksi yang hidup selama menit/jam.

Solusi: Pusher Channels sebagai WebSocket Proxy

Client Browser ←—— WebSocket Persistent ———→ Pusher Server

↑

(klik kirim pesan)

(trigger event)

|

|

HTTP POST /api/chat/send → Vercel Function

(simpan ke DB,

panggil Pusher REST API,

lalu mati – done)

Pusher yang 'mengingat' siapa yang sedang subscribe ke channel apa.

Vercel Function hanya perlu hidup selama 1 HTTP request.

7.2.1 Alur Sender — Memulai dan Menggunakan Sesi Chat

#	Aksi	Detail & Business Logic
1	Pilih Menu Curhat	Sender klik menu 'Cerita / Curhat' di /chat. Server Component merender halaman dengan daftar sesi chat yang pernah ada (dari tabel chat_sessions WHERE user_id = sender). Di bagian atas, ditampilkan status ketersediaan admin yang diambil dari tabel admin_profiles (admin mana yang saat ini Online).
2	Buat Sesi Baru	Sender klik 'Mulai Chat Baru'. Server Action createChatSession() dipanggil. Logic: (a) cek apakah Sender sudah punya OPEN session yang belum di-close (jika ya, redirect ke session yang ada — mencegah spam session), (b) jika tidak ada, INSERT ke tabel chat_sessions dengan status OPEN dan user_id Sender, (c) trigger Pusher event 'new-chat-session' ke channel admin.
3	Masuk ke Ruang Chat	Sender di-redirect ke /chat/[sessionId]. Server Component mengambil riwayat pesan dari tabel chat_messages dan merender di client. Client-side React component subscribe ke Pusher private channel 'private-chat-[sessionId]' untuk menerima pesan baru secara real-time.
4	Kirim Pesan	Sender mengetik pesan dan klik 'Kirim'. POST request dikirim ke API Route /api/chat/send dengan payload { sessionId, content }. Server: (a) validasi Sender adalah member dari sessionId (RLS), (b) INSERT ke chat_messages, (c) panggil Pusher REST API untuk

#	Aksi	Detail & Business Logic
5	Lihat Status Pesan	trigger event 'new-message' ke channel 'private-chat-[sessionId]', (d) cek apakah ada admin yang subscribe — jika tidak (offline), tambahkan ke antrian email notifikasi. Setiap pesan yang berhasil dikirim menampilkan ✓ (terkirim ke server). Saat admin membuka ruang chat yang sama, API Route /api/chat/[id]/read dipanggil otomatis, yang: (a) UPDATE chat_messages SET is_read=true, read_at=NOW() untuk semua pesan yang belum dibaca, (b) trigger Pusher event 'messages-read' ke channel sender. Indikator berubah menjadi ✓✓ di sisi Sender.
6	Terima Balasan Real-time	Saat admin membalas, pesan muncul real-time di ruang chat Sender via Pusher subscription — tanpa perlu refresh halaman. Jika Sender menutup browser dan kembali nanti, semua riwayat chat masih tersedia karena tersimpan di database (tidak bergantung pada Pusher event history yang hanya 30 detik).

7.2.2 Penanganan Skenario Offline (Asynchronous Chat)

Penanganan Mode Offline — Tidak Ada Pesan yang Hilang	
SKENARIO A: Admin offline saat Sender mengirim pesan	
1.	Sender kirim pesan → POST /api/chat/send
2.	Server: INSERT pesan ke DB (BERHASIL — tidak peduli admin online/offline)
3.	Server: trigger Pusher event → Pusher coba deliver ke admin channel subscribers
4.	Karena admin offline, tidak ada subscriber → event expired setelah 30 detik
5.	Server (sebelum return): cek admin_profiles.availability_status
	→ Jika OFFLINE/AWAY: INSERT ke tabel notifications (type='NEW_CHAT_MESSAGE')
	→ Resend API mengirim email ke admin: 'Ada pesan baru dari [Nama/Anonim]'
6.	Sender tetap melihat ✓ (terkirim ke server) — karena memang sudah tersimpan di DB
SKENARIO B: Sender offline saat admin membalas	
1.	Admin kirim balasan → POST /api/chat/send (admin juga pakai endpoint yang sama)
2.	Server: INSERT pesan ke DB, trigger Pusher event ke 'private-chat-[sessionId]'
3.	Sender tidak subscribe (offline) → event expired

4. Server: INSERT ke tabel notifications Sender (type='NEW_CHAT_REPLY')
5. Resend API kirim email ke Sender: 'Admin PSDM telah membalas pesan Anda'
6. Saat Sender buka /chat/[sessionId] lagi: Server Component fetch ulang semua pesan dari DB – tidak ada yang hilang.

7.2.3 Alur Admin — Menangani Sesi Chat

#	Aksi	Detail & Business Logic
1	Terima Notifikasi Chat	Admin menerima: (a) pop-up real-time di dashboard via Pusher event 'new-chat-session', (b) badge counter di menu 'Chat' bertambah, (c) email notifikasi jika admin sedang offline saat pesan pertama masuk.
2	Assign Diri ke Session	Admin membuka /admin/chat dan melihat daftar session yang OPEN dan belum di-assign. Admin klik 'Ambil Kasus' yang memicu Server Action assignAdminToSession(): UPDATE chat_sessions SET assigned_admin_id = adminId. Session terhapus dari daftar 'unassigned' admin lain — satu session hanya ditangani satu admin.
3	Buka Ruang Chat	Admin di-redirect ke /admin/chat/[sessionId]. Server Component fetch riwayat pesan dan render. Admin subscribe ke Pusher channel 'private-chat-[sessionId]'. Semua pesan yang belum dibaca otomatis di-mark sebagai read (trigger event 'messages-read' ke Sender).
4	Balas Pesan	Admin mengetik dan kirim balasan. Mekanisme pengiriman identik dengan Sender: POST ke /api/chat/send dengan sessionId. Server menyimpan ke DB dan trigger Pusher event. Perbedaan: API Route memvalidasi bahwa sender_id adalah assigned_admin dari session tersebut.
5	Update Status Availability	Admin dapat mengubah status ketersediaan (Online / Away / Offline) dari dropdown di navbar. PATCH /api/admin/availability memperbarui tabel admin_profiles.availability_status. Perubahan real-time di-broadcast via Pusher ke halaman /chat Sender, sehingga indikator status admin selalu terkini.
6	Tutup Sesi Chat	Setelah percakapan selesai, admin klik 'Tutup Sesi'. Server Action closeChatSession(): (a) UPDATE chat_sessions SET status='CLOSED', closed_at=NOW(), (b) trigger Pusher event 'session-closed' ke channel Sender, (c) kirim email notifikasi ke Sender bahwa sesi telah ditutup. Session menjadi read-only arsip. Sender masih bisa membaca riwayat di /chat/[sessionId] tetapi tidak bisa mengirim pesan baru.

7.3 Alur Permintaan Janji Temu

Fitur janji temu dirancang untuk meminimalkan kompleksitas teknis sambil tetap menyediakan pencatatan yang cukup untuk keperluan rekap akhir periode. Tidak ada sistem kalender atau booking slot waktu — alur berakhir di WhatsApp untuk menjaga kesederhanaan dan kemudahan adopsi.

7.3.1 Business Logic Keamanan Nomor WhatsApp

Keamanan Nomor WhatsApp Admin	
Masalah: Nomor WhatsApp admin adalah data sensitif yang tidak boleh terekspos	
Jika nomor disimpan di frontend atau dikembalikan mentah dari API, siapapun bisa inspect network tab dan mendapatkan nomor admin.	
Solusi: Server-side URL Generation	
1. wa_number di tabel admin_profiles disimpan dalam format TERENKRIPSI (AES-256-GCM dengan ENCRYPTION_KEY dari Vercel Environment Variables) 2. Supabase RLS: kolom wa_number_encrypted TIDAK pernah dikembalikan ke client meski ada yang query langsung (column-level security). 3. Saat Sender klik nama admin: → POST /api/appointments/wa-link { targetAdminId } → Server Action: decrypt wa_number, format URL, INSERT ke tabel appointments → Return: { redirectUrl: 'https://wa.me/62XXXX?text=...' } – bukan nomornya 4. Client menerima URL jadi dan langsung redirect. Nomor TIDAK pernah muncul di response, di source code, atau di browser history.	

7.3.2 Alur Lengkap Permintaan Janji Temu

#	Aksi	Detail & Business Logic
1	Buka Menu Janji Temu	Sender mengakses /appointments. Server Component fetch dari tabel admin_profiles JOIN users WHERE role IN (ADMIN, SUPER_ADMIN). Data yang dikembalikan: nama, jabatan_display, availability_status, dan avatar_url. Kolom wa_number_encrypted TIDAK ikut di-fetch (dipilih secara eksplisit di query SELECT).
2	Lihat Direktori Admin	Halaman menampilkan card untuk setiap admin: nama, jabatan, dan badge status (Online ditampilkan hijau, Away kuning, Offline abu-abu). Data di-cache di Vercel KV dengan TTL 60 detik untuk mengurangi beban query

#	Aksi	Detail & Business Logic
3	Pilih Admin yang Dituju	ke Supabase. Cache di-invalidate saat ada perubahan availability. Sender klik card admin. Business logic: sistem memeriksa apakah Sender sudah pernah membuat appointment ke admin yang sama dalam 24 jam terakhir. Jika ya, tampilkan warning 'Anda baru saja menghubungi admin ini. Pastikan janji temu sebelumnya sudah selesai.' Jika tidak, lanjutkan ke langkah berikutnya.
4	Generate WA Link (Server-side)	Sender klik 'Hubungi via WhatsApp'. POST request ke /api/appointments/wa-link dengan { targetAdminId }. Server: (a) decrypt wa_number dari DB, (b) compose template: 'Halo Kak [nama admin], saya [nama sender] dari [biro] ingin mengajukan janji temu melalui Halo PSDM ARSC.' (c) URL-encode template, (d) INSERT record baru ke tabel appointments untuk logging, (e) return { redirectUrl }.
5	Redirect ke WhatsApp	Client menerima redirectUrl dan melakukan window.open(redirectUrl, '_blank'). Browser membuka WhatsApp Web atau aplikasi WhatsApp. Pesan template sudah terisi otomatis di kolom teks — Sender tinggal klik 'Kirim'. Seluruh komunikasi selanjutnya terjadi di WhatsApp di luar sistem.
6	Logging untuk Rekap	Record di tabel appointments mencatat: user_id Sender, target_admin_id, dan timestamp. Data ini digunakan untuk rekap akhir periode: 'Admin X menerima Y permintaan janji temu.' Tidak ada data percakapan WhatsApp yang diambil — sistem hanya mencatat intent, bukan isi komunikasi.

8. Struktur API Routes (Serverless Functions)

Seluruh API Routes di bawah berjalan sebagai Vercel Serverless Functions. Setiap route memvalidasi session dan role sebelum mengakses database.

8.1 Authentication

Method	Endpoint	Fungsi	Keterangan
POST	/api/auth/[...nextauth]	Login, logout, refresh token	NextAuth.js v5 handler
GET	/api/auth/session	Cek status session aktif	NextAuth.js built-in

8.2 Reports (Laporan)

Method	Endpoint	Fungsi	Keterangan
GET	/api/reports	List laporan (admin: semua, sender: milik sendiri)	RLS otomatis filter
POST	/api/reports	Submit laporan baru	Server Action wrapper
GET	/api/reports/[caseId]	Detail laporan + status history	JOIN dengan history table
PATCH	/api/reports/[caseId]/status	Update status (admin only)	Validasi transisi + notify
PATCH	/api/reports/[caseId]/notes	Update catatan internal (admin only)	Tidak di-expose ke sender

8.3 Chat

Method	Endpoint	Fungsi	Keterangan
GET	/api/chat/sessions	List sesi chat aktif (role-based)	RLS filter per role
POST	/api/chat/sessions	Buat sesi chat baru (sender)	Cek existing open session
GET	/api/chat/sessions/[id]	Riwayat pesan sesi	Validasi member session
POST	/api/chat/send	Kirim pesan (sender & admin)	Save DB + Pusher trigger
PATCH	/api/chat/sessions/[id]/read	Mark pesan sebagai dibaca	Bulk update + Pusher event
PATCH	/api/chat/sessions/[id]/assign	Assign admin ke session (admin)	Update assigned_admin_id
PATCH	/api/chat/sessions/[id]/close	Tutup sesi chat (admin)	Update status + notify sender

8.4 Appointments & Admin

Method	Endpoint	Fungsi	Keterangan
GET	/api/appointments/admins	Daftar admin tersedia (cached KV)	Exclude wa_number
POST	/api/appointments/wa-link	Generate WA redirect URL (secure)	Decrypt + log + return URL

Method	Endpoint	Fungsi	Keterangan
PATCH	/api/admin/availability	Update status ketersediaan admin	Update + Pusher broadcast

8.5 Notifications & Pusher

Method	Endpoint	Fungsi	Keterangan
GET	/api/notifications	List notifikasi user aktif (unread)	Filter by user_id
PATCH	/api/notifications/read-all	Mark semua notif sebagai dibaca	Bulk UPDATE DB
POST	/api/pusher/auth	Authenticate Pusher private channel	Validasi session + sign

9. Sistem Notifikasi

Sistem notifikasi menggunakan strategi berlapis: Pusher Channels untuk pengiriman real-time (ketika pengguna sedang online), dan Resend email sebagai fallback (ketika pengguna offline). Setiap notifikasi juga disimpan ke tabel notifications di database untuk ditampilkan saat pengguna kembali login.

9.1 Matrix Notifikasi

Event Trigger	Notifikasi ke Admin	Notifikasi ke Sender
Laporan baru di-submit	Pusher pop-up + email (jika offline) + badge +1	Email konfirmasi + Case ID
Status laporan diubah admin	—	Pusher real-time (jika online) + email + notif in-app
Admin buka status NEEDS_CLARIFICATION	—	Pusher + email: 'Chat klarifikasi telah dibuka'
Laporan di-close (DONE)	—	Pusher + email: 'Kasus Anda telah diselesaikan'
Chat baru dimulai oleh Sender	Pusher pop-up + email (jika offline) + badge +1	—
Admin membalas pesan chat	—	Pusher real-time (jika online) + email (jika offline)
Sender mengirim pesan chat	Pusher real-time (jika online) + email (jika offline)	—

Event Trigger	Notifikasi ke Admin	Notifikasi ke Sender
Session chat ditutup admin	—	Pusher + email: 'Sesi chat telah ditutup'
Permintaan janji temu baru	Badge +1 di menu Appointments	—

9.2 Pusher Channel Architecture

Channel Name	Type	Subscriber	Events yang Dikirim
private-admin-global	Private	Semua admin aktif	new-report, new-chat-session, appointment-request
private-chat-[sessionId]	Private	Sender + assigned admin	new-message, messages-read, session-closed
private-user-[userId]	Private	Individual sender	status-updated, report-done, notification
private-admin-availability	Private	Semua sender di halaman /chat	availability-changed (update badge Online/Offline)

10. Strategi Deployment di Vercel

10.1 Environment Variables

Seluruh konfigurasi sensitif disimpan di Vercel Dashboard sebagai Environment Variables, tidak pernah di-commit ke repository:

Variable	Digunakan Untuk
DATABASE_URL	Supabase connection string dengan Supabase pooler (mode: transaction, untuk serverless)
DIRECT_URL	Supabase direct connection string (untuk Drizzle migrations saja, tidak dipakai di runtime)
NEXTAUTH_SECRET	JWT signing secret untuk NextAuth.js session
NEXTAUTH_URL	Base URL produksi (https://halo-psdm.vercel.app)
PUSHER_APP_ID, PUSHER_KEY, PUSHER_SECRET, PUSHER_CLUSTER	Kredensial Pusher untuk server SDK dan client SDK
RESEND_API_KEY	API key untuk pengiriman email transaksional via Resend

Variable	Digunakan Untuk
ENCRYPTION_KEY	AES-256-GCM key untuk enkripsi/dekripsi nomor WhatsApp di database
NEXT_PUBLIC_PUSHER_KEY, NEXT_PUBLIC_PUSHER_CLUSTER	Pusher client credentials (boleh public, tidak sensitif)

10.2 Supabase + Vercel Connection (Serverless-Specific)

Konfigurasi Koneksi Supabase untuk Serverless
Masalah: PostgreSQL standar tidak cocok untuk serverless
Setiap Vercel Serverless Function invocation membuka koneksi DB baru. Untuk high-traffic, ini menyebabkan 'connection pool exhausted'.
Solusi: Supabase Supavisor (Connection Pooler)
DATABASE_URL menggunakan connection string Supavisor (port 6543, mode: transaction): <pre>postgresql://[user].[project-ref]:[password]@aws-0-[region].pooler.supabase.com:6543/postgres</pre>
Supavisor mengelola pool koneksi ke PostgreSQL. Setiap Serverless Function connect ke Supavisor (ringan), bukan langsung ke Postgres. Koneksi ke Postgres yang sebenarnya direcycle oleh Supavisor antar invocations.
DIRECT_URL (port 5432) hanya untuk Drizzle migrations (bun run db:migrate), yang dijalankan sekali saja – bukan saat runtime.

10.3 CI/CD Pipeline

#	Aksi	Detail & Business Logic
1	Push ke GitHub	Developer push kode ke branch feature/*. Pull Request dibuat ke branch main.
2	Preview Deployment	Vercel otomatis membuat Preview Deployment dengan URL unik (https://halo-psdm-[branch]-[hash].vercel.app). Cocok untuk review dan testing fitur baru sebelum merge.

#	Aksi	Detail & Business Logic
3	bun install & build	Vercel menjalankan 'bun install' lalu 'bun run build' (next build). Bun jauh lebih cepat dari npm untuk proses ini — mempercepat deploy time.
4	Database Migration (Opsional)	Jika ada perubahan schema, 'bun run db:migrate' dijalankan via GitHub Actions sebelum merge ke main, menggunakan DIRECT_URL (koneksi langsung ke Supabase).
5	Production Deploy	Merge ke main trigger Production Deployment. Vercel melakukan atomic swap: traffic dialihkan ke deployment baru secara instan tanpa downtime. ISR cache di-invalidate otomatis.

10.4 Pertimbangan Serverless Constraints

Constraint Vercel	Dampak	Solusi yang Diterapkan
Execution timeout 10 detik (Hobby) / 60 detik (Pro)	API Route yang lambat akan timeout	Semua query DB dioptimasi dengan index. Operasi berat (email) dijadikan fire-and-forget.
Tidak ada persistent in-memory state	Tidak bisa menyimpan cache di memory function	State di Supabase DB (persisten) atau Vercel KV (cache). Pusher untuk WebSocket state.
Cold start latency	Request pertama setelah idle lebih lambat	Server Components di-cache dengan ISR. Drizzle ORM lebih ringan dari Prisma (cold start lebih cepat).
Max request body 4.5 MB	Batasan ukuran upload file	File upload (jika ada) dikirim langsung ke Supabase Storage dari client, tidak melalui API Route.

11. Monitoring, Evaluasi, dan Rekap Akhir Periode

11.1 Dashboard Monitoring Real-time Admin

Dashboard admin menampilkan panel monitoring dengan widget-widjet berikut:

Widget	Data yang Ditampilkan	Sumber Data
Ringkasan Kasus	Total laporan, aktif, selesai, pending > 48 jam	Supabase: COUNT queries
Distribusi Status	Pie chart RECEIVED/IN_PROGRESS/NEEDS_CLARIFICATION/DONE	Supabase: GROUP BY status

Widget	Data yang Ditampilkan	Sumber Data
Response Time Rata-rata	Rata-rata waktu dari submit hingga status pertama diupdate	Supabase: AVG(first_status_update - created_at)
Top Kategori Masalah	Bar chart 5 kategori terbanyak dilaporkan	Supabase: GROUP BY category ORDER BY COUNT
Chat Aktif	Jumlah sesi OPEN yang belum di-assign	Supabase: COUNT chat_sessions WHERE status=OPEN

11.2 Rekap Akhir Periode

Pada akhir periode, admin Super Admin dapat mengunduh rekap otomatis dalam format CSV dari dashboard. Query rekap meliputi:

- Total laporan per kategori dan per biro/bidang — untuk identifikasi area yang paling banyak memiliki masalah.
- Average dan median response time per admin — untuk evaluasi kinerja tim PSDM.
- Total sesi chat, rata-rata durasi, dan rata-rata jumlah pesan per sesi.
- Total permintaan janji temu per admin yang dituju.
- Tren laporan per bulan — untuk melihat apakah ada lonjakan masalah di periode tertentu.

12. Pertimbangan Keamanan dan Privasi

Area Keamanan	Implementasi
Enkripsi Data in-transit	HTTPS enforced oleh Vercel untuk semua traffic. Tidak ada HTTP plain text.
Enkripsi Data at-rest	Supabase PostgreSQL mengenkripsi data at-rest secara default.
Autentikasi Session	NextAuth.js JWT disimpan di cookie HttpOnly dan Secure — tidak dapat diakses oleh JavaScript client.
Row Level Security (RLS)	Supabase RLS diterapkan di semua tabel. Sender tidak bisa membaca data pengguna lain meski query langsung ke Supabase.
Column-level Security	Kolom wa_number_encrypted di admin_profiles tidak pernah dikembalikan ke client melalui Supabase query — hanya diakses via Server Function dengan dekripsi eksplisit.

Area Keamanan	Implementasi
Input Validation	Dua lapis: Zod di client (UX feedback cepat) dan Zod di Server Action/API Route (keamanan). SQL injection tidak mungkin karena Drizzle ORM menggunakan parameterized queries.
CSRF Protection	Next.js Server Actions memiliki CSRF protection built-in. API Routes sensitif menggunakan token verification tambahan.
Rate Limiting	Vercel KV (Redis) digunakan untuk rate limiting: max 5 laporan per user per 24 jam, max 100 chat messages per session per jam.
Pusher Private Channels	Channel chat menggunakan Pusher Private Channels — hanya client yang telah diautentikasi server-side via /api/pusher/auth yang dapat subscribe.
Anonimitas Laporan	Jika is_anonymous=true: nama sender ditampilkan sebagai 'Anggota Anonim' ke admin level ADMIN. Hanya SUPER_ADMIN yang dapat reveal identitas sebenarnya untuk keperluan audit.

13. Roadmap Implementasi

Fase	Durasi	Deliverable Utama
Fase 1: Setup & Foundation	Minggu 1–2	Init Next.js 16 + Bun, konfigurasi Supabase project, schema migration Drizzle, implementasi NextAuth.js, deploy skeleton ke Vercel, env vars setup.
Fase 2: Fitur Pelaporan	Minggu 3–4	Form laporan + Server Action, Case ID generation, status lifecycle, email via Resend, Pusher setup, admin laporan dashboard.
Fase 3: Live Chat	Minggu 5–6	Pusher Channels integration, chat UI, async messaging, read receipts, availability status, notifikasi email fallback.
Fase 4: Janji Temu	Minggu 7	Direktori admin, enkripsi WA number, WhatsApp redirect, appointment logging.
Fase 5: Polish & Testing	Minggu 8	Mobile responsiveness, edge case handling, rate limiting, security review, User Acceptance Testing bersama PH PSDM.
Fase 6: Launch & Training	Minggu 9	Production deployment, training admin PSDM, dokumentasi user guide, sosialisasi ke seluruh anggota ARSC.

14. Penutup

Concept paper ini memaparkan rancangan lengkap Sistem Halo PSDM berbasis website untuk ARSC periode 2025/2026. Dengan mengadopsi stack teknologi modern yang sepenuhnya serverless (Next.js 16, Bun, Vercel, Supabase, Pusher), sistem ini dirancang untuk menjadi solusi yang berkelanjutan: mudah di-maintain, biaya operasional minimal, dan dapat diwariskan ke pengurus periode berikutnya tanpa memerlukan pengelolaan infrastruktur server secara manual.

Tiga alur utama — Pelaporan, Live Chat, dan Janji Temu — telah dirancang secara detail dengan mempertimbangkan seluruh skenario pengguna, termasuk mode offline, transisi status kasus, assignment admin, dan keamanan data sensitif. Sistem notifikasi berlapis (Pusher real-time + email fallback + in-app notification) memastikan tidak ada laporan yang terlewat, sementara Supabase Row Level Security menjamin bahwa data setiap anggota terlindungi di level database.

Dokumen ini dimaksudkan sebagai panduan perencanaan yang hidup (living document) dan dapat direvisi sesuai dengan keputusan teknis yang berkembang selama proses implementasi.